NLP Preprocessing & Word2Vec

Structured Notes

Text Cleaning, Tokenization,

Stemming, Lemmatization, Embeddings

Contents

1 Why Text Cannot Be Fed Directly Into Neural Networks

Neural networks understand only numbers because they are built on matrix and linear algebra. For applying any algebraic or matrix operations, inputs must be numerical. Raw text has to be converted into a numerical representation before it can flow into any model.

Core Reason

Neural networks operate on matrices. Algebraic operations require numbers, so raw strings cannot be processed directly and must first be transformed into numerical form.

Raw text is also generally messy. It is not in a good format. Typical issues include:

- Upper case versus lower case inconsistency
- Extra spaces
- Punctuations
- Special characters
- Numbers

Machine learning models cannot interpret these variations correctly. For example, “machine”, “Machine”, and “machine-” would be treated as different words, which unnecessarily increases the vocabulary size.

Vocabulary Explosion

Variations in casing, spacing, punctuation and special characters inflate vocabulary size and introduce noise, which hurts downstream model learning.

2 Text Cleaning Pipeline

The first stage of any NLP workflow is a text cleaning pipeline. Just like numeric data preprocessing, text preprocessing is **task dependent**. There is a general pipeline, but based on the task some steps may be kept or skipped.

2.1 Standard Cleaning Steps

- Lowercasing all words
- Removing punctuations
- Removing special characters
- Removing numbers (optional)
- Removing extra spaces

2.2 Why Removing Numbers Is Optional

Numbers are not always noise. Sometimes numbers carry meaning. If a dataset contains a five star rating, removing the number loses the meaning. In that case the number can even be treated as a string and later converted into embeddings.

On the other hand, in a sentence like “Deep learning costs $1000 sometimes”, the number can be dropped because the context of high cost vs low cost is already conveyed by the surrounding words.

Always Think Before Preprocessing

Removing numbers can lose meaning. Text preprocessing is task dependent, so always think before applying a preprocessing step.

2.3 Analogy With Numeric Preprocessing

The same task dependence exists in numeric preprocessing. Missing numeric data is not always replaced with the mean. Mean is highly sensitive to outliers. If the dataset contains meaningful outliers, missing data cannot be replaced with mean and median or mode must be used instead.

2.4 Effect Of Cleaning On Vocabulary

After cleaning, all words are lowercased, numbers (where appropriate) are removed, punctuations are removed and special characters are removed. This reduces the vocabulary size. For example, "machine", "Machine" and "machine" are all treated as the same word. This improves model learning.

3 Tokenization

After the data has been cleaned, the next step of getting embeddings is to convert the text into **tokens**.

Token

A token is the smallest unit of text. It can be a word or a sub-word, not just a whitespace-separated chunk.

3.1 Two Options For Tokenization

There are two main options:

1. Use Python's `.split()` string function.
2. Use a tokenizer provided by an NLP library (the better way).

NLP libraries provide in-built tokenizers, in-built embeddings and convert everything in one go. To understand how that works internally, the manual steps are still worth following, because these are the same ideas used under the hood by tokenizers in BERT and GPT.

3.2 dot-split vs word_tokenize

Using `.split()` on a sentence leaves a word like "don't" as a single token "don't". The NLP-library tokenizer `word_tokenize` captures the contextual split and produces "do" and "n't" (i.e. "do not"), which is better contextual catching.

NLTK word_tokenize

```
from nltk.tokenize import word_tokenize

sentence = "I don't love NLP"
tokens_split = sentence.split()
tokens_nltk  = word_tokenize(sentence)

# tokens_split -> ["I", "don't", "love", "NLP"]
# tokens_nltk  -> ["I", "do", "n't", "love", "NLP"]
```

Listing 1: Tokenizing with NLTK word_tokenize

3.3 Why Tokenization Is Required

Models work on tokens, not on sentences. A simple split is not always sufficient because it ignores language rules and fails to capture the contextual meaning of contractions and similar patterns. `word_tokenize` is aware of these language rules and produces better contextual clarity.

4 Stop Word Removal

After tokenization, the next cleaning-like step (still task dependent) is removing stop words.

Stop Words

Stop words are very frequent connecting words that individually contain very little meaning. Removing them reduces vocabulary and noise while usually preserving most of the context.

For example, in "I love machine learning", removing connecting words leaves "love machine learning" which still carries almost the full sentiment. Similarly, "NLP is amazing isn't it" compresses to "NLP amazing".

Language Matters

Stop word lists are language specific. For English, pass the English list; for Chinese, the Chinese list; for Hindi, the Hindi list. The stop word module is multilingual.

An intuition: humans do not have to read a complete sentence to understand context. By reading a few of the keywords, the context can still be formed, even if the word order is not correct. That is also how NLP and transformer models work.

5 Stemming

After stop word removal, the next step is stemming.

Stemming

Stemming reduces a word to its root form using rule-based chopping. In NLTK this is done with the `PorterStemmer`.

Porter Stemmer

```python
from nltk.stem import PorterStemmer

stemmer = PorterStemmer()
stemmer.stem("machines")   # -> "machin"
stemmer.stem("amazing")    # -> "amaz"
stemmer.stem("learning")   # -> "learn"
```

Listing 2: Stemming with PorterStemmer

5.1 Problem With Stemming

Stemming sometimes produces words that do not exist in the language. "machines" becomes "machin" (the "e" is missing). "amazing" becomes "amaz" (again the "e" is missing). These are not valid dictionary words.

Stemming is fast but crude. It preserves the context loosely but drops trailing characters blindly, so "learning" also becomes "learn" and the "ING" information is lost.

6 Lemmatization

A better alternative to stemming is lemmatization.

Lemmatization

Lemmatization converts a word to its linguistically correct base form (the lemma). It uses dictionary knowledge rather than rule-based chopping.

Lemmatization is linguistically correct but is slightly slower than stemming in terms of memory and time processing. When the dataset is small and fast processing is needed, stemming may still be preferred.

6.1 Simple Lemmatization

Simple lemmatization preserves context better than stemming but can still over-normalize. Words like "learning" and "learn" may both collapse to "learn".

6.2 POS-Tag Aware Lemmatization

A better way of lemmatization is to use POS (part of speech) tags.

POS Tag

A POS tag is a dictionary-style label that marks a word as an adjective, verb, noun, adverb, etc. Lemmatization guided by POS tags produces more accurate root words.

With POS-tag aware lemmatization:

- In "I love machine learning", "learning" is converted to "learn".
- In "Learn NLP Masai", "learning" remains as "learning" to save the contextual meaning.

Another example set: given the words "running", "better", "studies":

- Simple lemmatization → "running", "better", "studies"
- POS-tag aware lemmatization → "run", "good", "study"

The second form is a more optimized version.

Preferred In Production

POS-aware lemmatization produces more meaningful root words, gives better linguistic normalization and improves embedding quality. It is preferred in most production NLP pipelines.

7 Observations From The Preprocessing Exercise

- Stop word removal drops less meaningful words.
- Stemming is fast but crude and can create non-existent words.
- Lemmatization is a little slower but linguistically correct.
- The choice between stemming and lemmatization depends on the task.
- POS-aware lemmatization produces more meaningful root words, better linguistic normalization and improved embedding quality.

8 Why Wrap Preprocessing In A Function

Preprocessing logic is wrapped into a function so it can be used both during training and inference. If preprocessing is inconsistent between these two stages, model performance degrades.

Consistency Rule

The same preprocessing function must be applied during training and during inference. Any

Technique	Strength	Weakness
`.split()`	Simple, no dependencies	Ignores language rules, bad on contractions
`word_tokenize`	Language-aware splits, contractions handled	Needs NLP library
Stop word removal	Reduces vocabulary, removes noise	Risk of losing context in short texts
Stemming	Fast, crude reduction	Produces non-dictionary words
Simple lemmatization	Linguistically correct	Can over-normalize similar words
POS-aware lemmatization	Best normalization, preserves context	Slightly slower, needs POS tagger

inconsistency degrades model performance.

9 From Toy Data To Real Customer Data

The same preprocessing pipeline is applied on a more realistic customer review dataset using pandas and `df.apply`. The raw review dataset is loaded into a DataFrame, and the cleaning / tokenization pipeline is mapped onto the review column.

A more realistic dataset brings in more noise: emojis, hashtags, URLs, mixed casing, spelling noise and tickets. When such noise is present, more preprocessing changes may be required, and stemming can be too aggressive. This is the kind of data used for three purposes:

1. Preprocessing
2. Embeddings
3. Deployment

The main place where people get stuck in an NLP project is the preprocessing pipeline.

10 Word2Vec Embeddings

After preprocessing, the next step is to convert tokens into embeddings.

Word2Vec

Word2Vec creates **dense vectors** for given words such that contextual meaning is not lost. Unlike one-hot encoding, which creates sparse vectors, Word2Vec places each word as a point in a high-dimensional space where words used in similar contexts have similar vector representations.

A classic property:

$$\text{king} - \text{man} + \text{woman} \approx \text{queen}$$

Each word becomes a dense vector. This helps the model understand similarity, context and meaning.

10.1 Mechanism Demo On Preprocessed Sentences

Word2Vec tokenizes sentences, builds a vocabulary and learns a dense vector for each word. A small demo is first run on a short list of already-preprocessed sentences before scaling to the real dataset.

Word2Vec Demo Model

```python
from gensim.models import Word2Vec

# sentences is a list of pre-processed token lists
demo_model = Word2Vec(
    sentences,
    vector_size=50,
    window=3,
    workers=4
)

print(demo_model.wv["NLP"].shape)  # -> (50,)
print(demo_model.wv.most_similar("learning"))
```

Listing 3: Training a small Word2Vec demo model

10.2 Key Hyperparameters

- **vector_size**: The size of the dense vector per word (for example 50). A hyperparameter to tune. More dimensions can capture more information.
- **window**: The contextual window size. A window of 3 means three words are looked at to the left / right, then the window shifts — conceptually similar to a sliding window (and to the stride idea in pooling).
- **workers**: Number of parallel processes. Depends on the CPU cores available and is used for parallel processing in the background.

10.3 Vocabulary And Vector Shape

After training the demo model, a vocabulary is built from the unique tokens. The vocabulary is a set, so only unique words matter and the order does not matter. A single word like "NLP" is then represented by a dense vector of shape `(50,)` when `vector_size=50`.

10.4 Normalization And Dense Vectors

Values in the dense vectors lie in a narrow range around zero (both negative and positive), which corresponds to a Gaussian-style normalization. Word2Vec uses a Gaussian-fit style normalization to turn words into vectors.

Gaussian Normalization

Word2Vec uses a Gaussian function style normalization to convert words into vectors. This is numeric normalization, not linguistic normalization.

10.5 Similarity Via Co-occurrence

Similarity between words is computed using the co-occurrence structure of the corpus. A co-occurrence matrix is built, and from that matrix similar words end up with similar vectors. The model's `most_similar` function returns the top similar words in **descending order** of similarity, and the reported numbers are similarity scores based on cosine similarity.

$$\text{cosine_sim}(\vec{u}, \vec{v}) = \frac{\vec{u} \cdot \vec{v}}{\|\vec{u}\| \, \|\vec{v}\|}$$

Higher similarity score means more similar.

11 Reusable Word2Vec Training Function

Once the mechanism is understood, the training process is wrapped into a reusable function.

Reusable Trainer

```python
from gensim.models import Word2Vec

def train_word2vec(sentences, vector_size=50, window=3, workers=4):
    model = Word2Vec(
        sentences,
        vector_size=vector_size,
        window=window,
        workers=workers
    )
    return model
```

Listing 4: Reusable Word2Vec training function

Benefits:

- Same logic can be reused.
- Easier experimentation.
- Cleaner pipeline.
- Easier deployment and scaling.

Word2Vec Is Itself A Model

Word2Vec is itself a model that trains and calculates similarity scores before producing embeddings. Internally it uses training schemes such as Skip-gram to learn similar embeddings.

12 Applying Word2Vec On The Real Dataset

The reusable function is then applied on the processed tokens of the real customer review dataset (`df["processed_tokens"]`). After training, the resulting model exposes a vocabulary, and words like `delivery`, `payment`, `support` and `quality` can be inspected.

For each queried word, the code checks whether the word is in the vocabulary, prints its vector shape, and prints the most similar words. For example:

- **delivery** → receive, chat, link, ever, ID, helpful
- **payment** → service, checkout, soft
- **support** → pixel, shoe, staff, email
- **quality** → delivery, complaint

This shows that the model has learned domain-specific relationships from the dataset itself and that the same Word2Vec logic now works on real customer-style text data. Embeddings have moved from concept to a practical NLP pipeline.

13 Concept Check

- **What does Word2Vec take as input?** Tokens, not raw words directly.
- **What is the output for each word?** A dense vector (an embedding).
- **Why do similar-context words become close in vector space?** Because they end up with similar vector representations, so the distance between them is small.
- **Why wrap Word2Vec training into a function?** Because it is reusable and faster.
- **Why can embeddings differ across domains?** Because the context differs across domains.
- **Why may results on a small dataset be weak?** Because there is less context to find similarity, which leads to underfitting. Word2Vec needs enough context and data.

14 Skip-gram vs CBOW And Visualization

Two standard training schemes for Word2Vec are **Skip-gram** and **Continuous Bag of Words (CBOW)**. These are the internal training strategies used to learn the embeddings.

After embeddings are learned, they can be visualized using **t-SNE**:

- **Why t-SNE?** For dimensionality reduction, to visualize high-dimensional embeddings.
- **Does t-SNE preserve exact distances?** No. It approximates the structure because the dimensionality is being reduced.
- **Why do some clusters overlap?** Because the dataset is small and contexts may overlap.

On the customer review data, clusters such as a delivery cluster, a payment cluster and a support cluster can be observed, showing that Word2Vec has captured semantic relationships.

15 Where Preprocessing And Embeddings Fit In The NLP Pipeline

The preprocessing pipeline plus embeddings form the **first part** of an NLP project — conceptually the **encoder** part. After this, the dense vectors are passed to an RNN, a transformer or a GPT-based model, and finally to a decoder to classify or predict the next word.

Encoder Role

The encoder creates embeddings from input tokens and stores them in a vector space. These embeddings are then consumed by the decoder (or a downstream model) to perform the actual task.

The high-level flow learned here is:

1. Preprocessing pipeline.
2. Similarity index via embeddings.
3. Downstream model (RNN / transformer / GPT) + decoder.

16 Summary

- Neural networks only understand numbers; raw text must first go through a preprocessing pipeline.
- Text cleaning handles case, punctuation, special characters, numbers (optional) and extra spaces, and reduces vocabulary size.
- Tokenization breaks sentences into tokens; `word_tokenize` is better than `.split()` because it is language-aware.
- Stop word removal drops connecting words that carry little individual meaning; the language of the stop word list must match the data.
- Stemming is fast but crude and can create non-words; lemmatization is linguistically correct; POS-aware lemmatization is the preferred production choice.
- Preprocessing is task dependent and must be consistent across training and inference.
- Word2Vec converts tokens into dense vectors using a Gaussian-style normalization, and similarity is computed from co-occurrence structure via cosine similarity.
- Key Word2Vec hyperparameters are `vector_size`, `window` and `workers`.
- Wrapping Word2Vec in a reusable function enables cleaner experimentation and deployment.
- On real customer review data, Word2Vec learns domain-specific clusters such as delivery, payment and support.
- Skip-gram and CBOW are the standard training schemes; t-SNE is used to visualize embeddings but only approximates distances.
- Preprocessing plus embeddings form the encoder side of the NLP pipeline, feeding an RNN, transformer or GPT-based decoder for the final task.